

Fall 2026 visualization club schedule

- **09/25** — Intro to Godot for 3D Visualization - 1 (*Ping Luo*)
- **10/02** — Intro to Godot for 3D Visualization - 2 (*Ping Luo*)
- **10/09** — How to configure a VR headset (*Peter Yu*)
- **10/16** — Intro to Unity (*Alex Forrence*)
- **10/23** — TBD
- **10/30** — No meeting
- **11/06** — Intro to Blender (*Cliona Kelly*)
- **11/13** — No meeting
- **11/20** — No meeting
- **11/27** — GPT-6 Astra for Interactive 3D World Building (*Ziqing Shi*)
VLM for medical video processing (*Ziqing Shi*)
- **12/04** — TBD

Introduction to Godot 4 for 3D Visualization

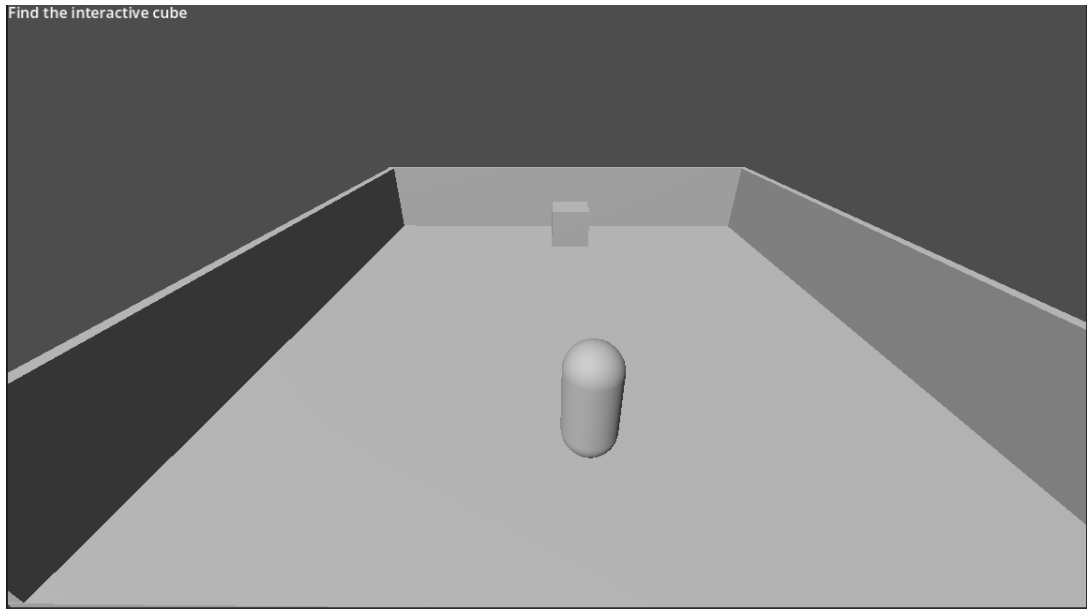
WTI Visualization Club

09/25/2026

What we will build

A tiny interactive 3D world

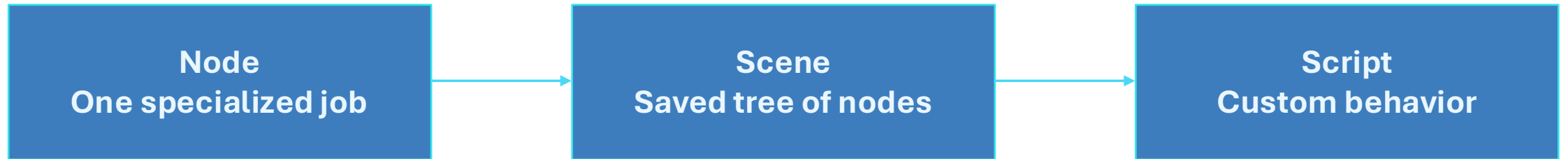
Use Godot's built-in structures only



- Walk around a simple room
- Collide with floor and walls
- Approach a cube
- Trigger a signal and update the UI

Core Concepts

Three basic ideas in Godot



The workshop repeatedly cycles through these three ideas.

specialized pieces

reusable assemblies

behavior over time

Nodes

A node does one specific job

Node3D

generic 3D object

MeshInstance3D

visible geometry

Label

screen text

Camera3D

what the player sees

DirectionalLight3D

provides light

CharacterBody3D

represents a script-
controlled character

CollisionShape3D

defines collision geometry

Area3D

detects object entering an area

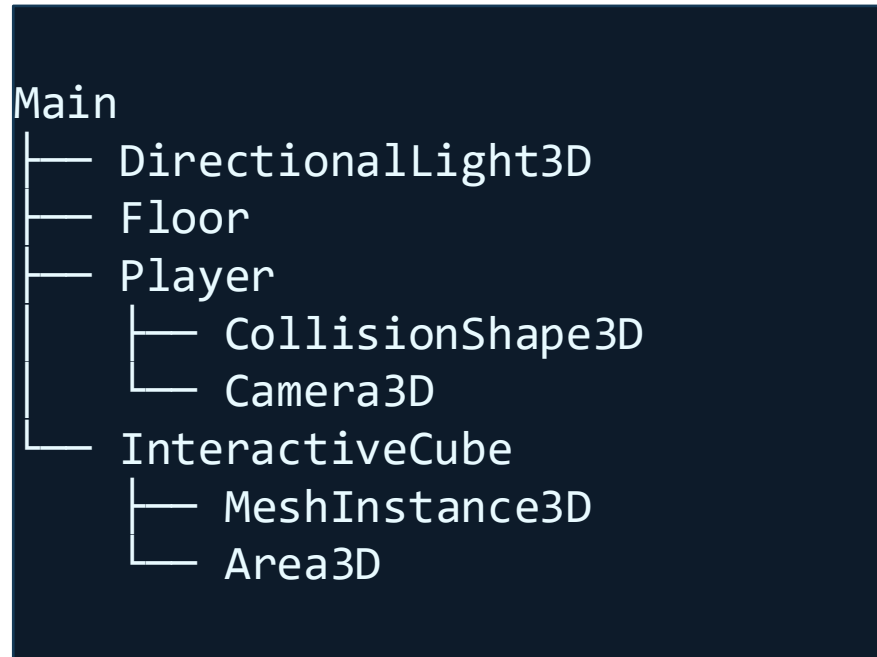
StaticBody3D

represents an immovable
physics object

Beginner rule: choose the node that matches the job.

Scene Tree

Nodes are organized hierarchically into a scene tree



The parent-child relationship matters.

- Lifetime/scene membership
- Node hierarchy/paths
- Transform inheritance

Ex: Player
 └─ Camera3D

If the Player moves, the camera moves with it.

Scenes

A scene is a saved tree of nodes.

```
Main
├── DirectionalLight3D
├── Floor
├── Player
│   ├── CollisionShape3D
│   └── Camera3D
├── InteractiveCube
│   ├── MeshInstance3D
│   └── Area3D
```

For example:

```
main.tscn
```

A scene can itself be placed inside another scene.

This makes scenes reusable.

Scripts

Nodes contain **properties** and **built-in behaviors**.

Scripts give them additional behavior.

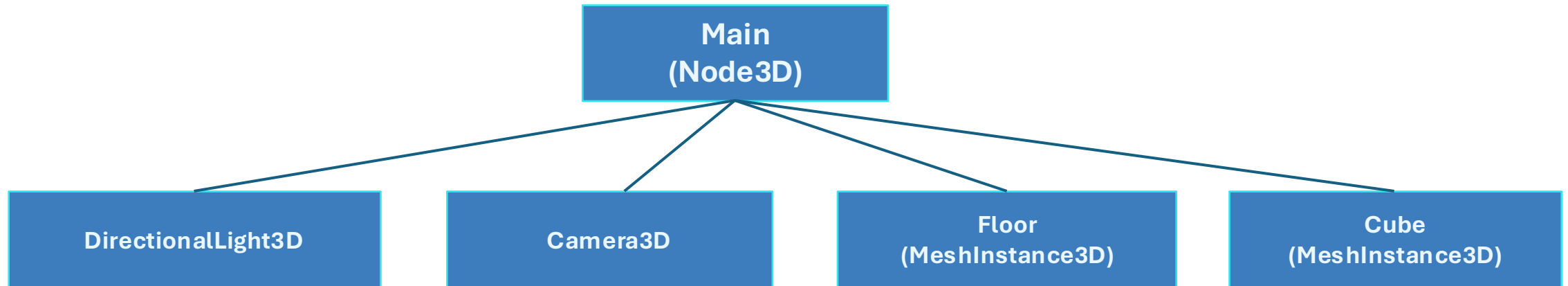
EX:



Our workshop will use **GDScript**, Godot's native scripting language.

Godot also supports C#

Build a 3D scene



Save it as **main.tscn**

3D Transform: position, rotation, scale

position, rotation, scale: **Vector3**

In code

```
position = Vector3(1, 2, 3)
rotation = Vector3(0, 1.57, 0)
scale = Vector3(2, 2, 2)
```

rotation is in radians

rotation_degree is in degree

These three editor properties describe the node relative to its parent

In the Inspector:

Floor
(MeshInstance3D)

BoxShape
position: 0.0, -0.1, 0.0
scale: 12, 0.2, 12

Cube
(MeshInstance3D)

BoxShape
position: 0.0, 0.5, -3
scale: 1.0, 1.0, 1.0

DirectionalLight3D

rotation: -50, -30, 0

Run project

Add behavior with GDScript

Attach a script to the cube

```
extends MeshInstance3D

func _ready() -> void:
    pass

func _process(delta: float) -> void:
    pass
```

- `extends`: inherits properties, methods, and behaviors of `MeshInstance3D`
- `_ready()`: is called once to set up code after the node is ready
- `_process(delta)`: is called every rendered frame.
- Godot passes `delta` to `_process()` automatically.

What is delta

delta = elapsed time since last frame

FPS=60 → $\text{delta} = 1/60 \approx 0.0167\text{s}$

FPS=120 → $\text{delta} = 1/120 \approx 0.0083\text{s}$

makes movement frame-rate independent

`rotation.y += rotation_speed * delta`

vs

`rotation.y += rotation_speed`

Make the cube rotate

```
extends MeshInstance3D

var rotation_speed := 1.0

func _process(delta: float) -> void:
    rotation.y += rotation_speed * delta
```

Run project

@export

```
extends MeshInstance3D

@export var rotation_speed := 1.0

func _process(delta: float) -> void:
    rotation.y += rotation_speed * delta
```

@export exposes a script variable to Godot's inspector.

- Easy experimentation
- Different instances can use different values
- Separate behavior with configuration
- Non-programmers can adjust behavior
- You can export more than numbers (see this later)

Check inspector for rotation_speed

Physics and Collision

Build a controllable player

```
Player (CharacterBody3D)
├── MeshInstance3D
├── CollisionShape3D
└── Camera3D
```

- MeshInstance3D gives visual shape
- CollisionShape3D gives physics volume
- Camera3D follows player as a child

**When the Player moves,
the Camera moves with it.**

In the Inspector:

Player
(CharacterBody3D)

position: (0, 0.9, 5)

MeshInstance3D

CapsuleShape3D
radius: 0.4
height: 1.6

CollisionShape3D

CapsuleShape3D
radius: 0.5
height: 1.6

Input Map

Project → Project Settings → Input Map

```
move_forward    → W  
move_backward   → S  
move_left       → A  
move_right      → D
```

Run project

Player movement

Attach player.gd

```
@export var speed: float = 5.0
func _physics_process(delta):
    var input_vector = Input.get_vector(
        "move_left", "move_right",
        "move_forward", "move_backward"
    )

    var direction = Vector3(
        input_vector.x, 0.0, input_vector.y
    )

    velocity.x = direction.x * speed
    velocity.z = direction.z * speed

    move_and_slide()
```

- Input gives direction
- Speed converts direction to velocity
- `move_and_slide()` asks physics to move
 - `move_and_slide()` is a built-in `CharacterBody3D` method
 - It uses the `CharacterBody3D.velocity` property automatically.
 - It moves the character using its `velocity`, detects collisions, and automatically slides the character along surfaces instead of stopping completely.

Understanding Input.get_vector()

```
@export var speed: float = 5.0
func _physics_process(delta):
    var input_vector = Input.get_vector(
        "move_left", "move_right",
        "move_forward", "move_backward"
    )

    var direction = Vector3(
        input_vector.x, 0.0, input_vector.y
    )

    velocity.x = direction.x * speed
    velocity.z = direction.z * speed

    move_and_slide()
```

```
Input.get_vector(
    "move_left",      -X
    "move_right",     +X
    "move_forward",   -Z
    "move_backward"   +Z
)
```

returns a Vector2.

W → (0, -1)

S → (0, 1)

A → (-1, 0)

D → (1, 0)

W + D → diagonal (0.707, -0.707)

Mesh vs Collision

MeshInstance3D
visible geometry

Controls what the user **sees**.

CollisionShape3D
physics volume

Controls what the physics engine **collides with**.

Use the right update loop

`_process(delta)`
visual / frame behavior

delta is related to FPS

`_physics_process(delta)`
movement / collision

delta is a fixed value,
unrelated to FPS

Simple rule: use physics loop when physics is involved.

Add a floor

```
Floor (StaticBody3D)  
├─ MeshInstance3D  
└─ CollisionShape3D
```

In the Inspector:

MeshInstance3D

BoxShape3D
X: 12
Y: 0.2
Z: 12

CollisionShape3D

BoxShape3D
X: 12
Y: 0.2
Z: 12

Add gravity on player

Modify player.gd

```
Player (CharacterBody3D)
├── MeshInstance3D
├── CollisionShape3D
└── Camera3D
```

```
@export var speed: float = 5.0
@export var gravity: float = 20.0
func _physics_process(delta):
    if not is_on_floor():
        velocity.y -= gravity * delta

    var input_vector = Input.get_vector(
        "move_left", "move_right",
        "move_forward", "move_backward"
    )

    var direction = Vector3(
        input_vector.x, 0.0, input_vector.y
    )

    velocity.x = direction.x * speed
    velocity.z = direction.z * speed

    move_and_slide()
```

Add walls

```
Wall (StaticBody3D)
├─ MeshInstance3D
└─ CollisionShape3D
```

Resue Wall

- Save as a scene: wall.tscn
- Drag wall.tscn from FileSystem to the 3D view
- Adjust position and rotation

In the Inspector:

Wall
StaticBody3D

position: (-5.9, 1.5, 0)

MeshInstance3D

BoxShape: x=0.2, y=3, z=12

CollisionShape3D

BoxShape: x=0.2, y=3, z=12

Wall1
StaticBody3D

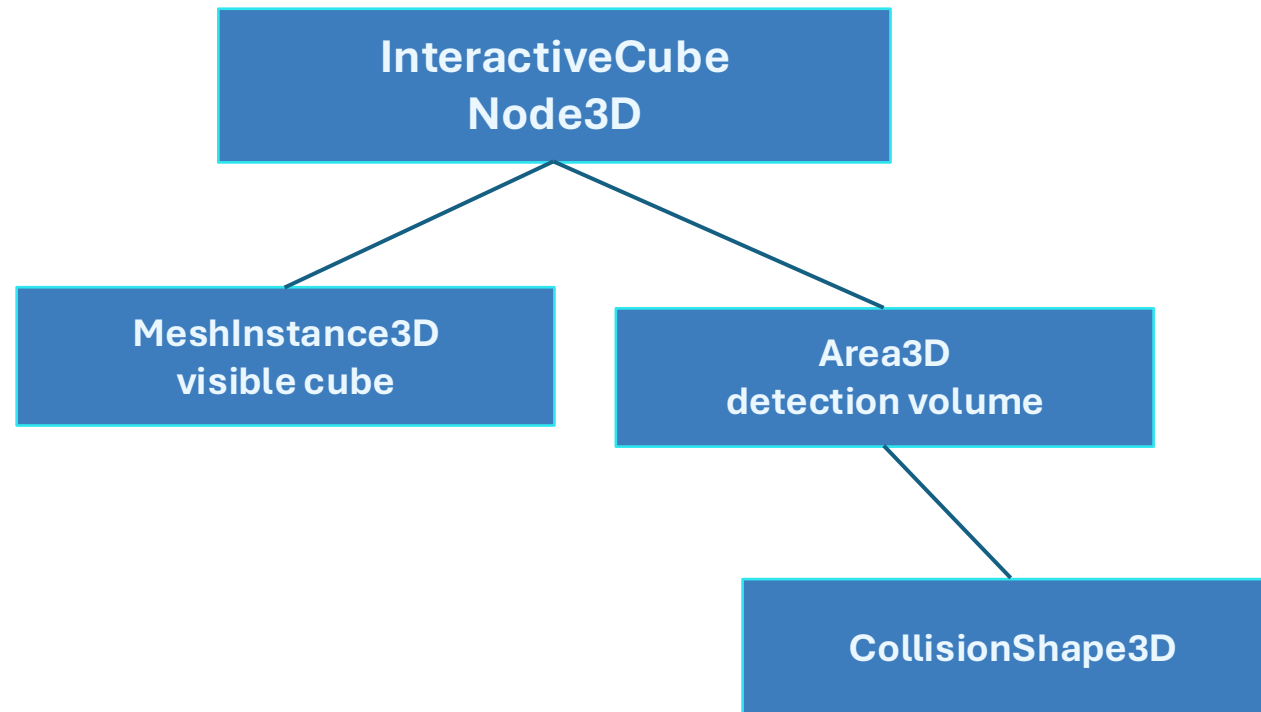
position: (5.9, 1.5, 0)

Wall2
StaticBody3D

position: (0, 1.5, -5.9)
rotation: (0, 90, 0)

Interaction

Create invisible detection zone



Create invisible detection zone

```
InteractiveCube (Node3D)  
├─ MeshInstance3D  
└─ Area3D  
    └─ CollisionShape3D
```

In the Inspector:

InteractiveCube
Node3D

MeshInstance3D

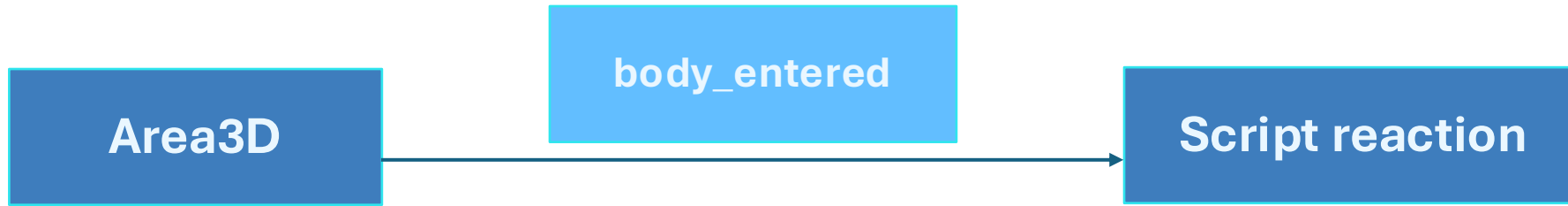
size: 1, 1, 1

Area3D

CollisionShape3D

BoxShape: size 2, 2, 2

Signals announce something happened



Decoupled event flow

Signals are central to Godot application architecture.

Other common signals include:

button pressed

timer timeout

body exited

animation finished

value changed

Change cube size

```
func _on_area_3d_body_entered(body):  
    if body.name == "Player":  
        $MeshInstance3D.scale = Vector3.ONE * 1.5  
        print("Player found the cube!")
```

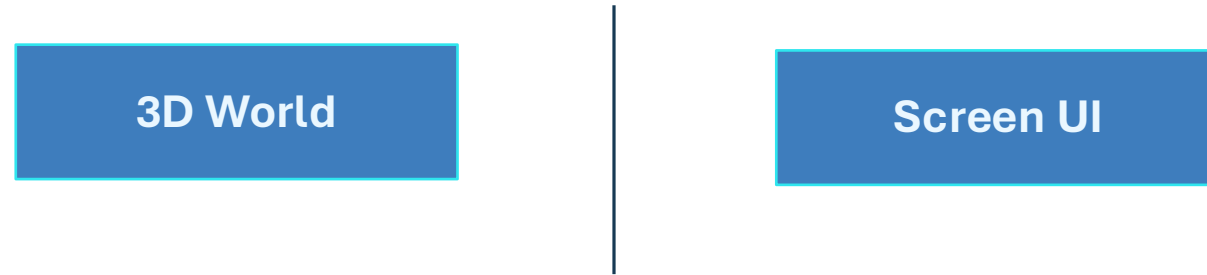
```
func _on_area_3d_body_exited(body):  
    if body.name == "Player":  
        $MeshInstance3D.scale = Vector3.ONE
```

Node paths

\$MeshInstance3D

\$Area3D/CollisionShape3D

User interface (CanvasLayer)



- World-space objects move with the camera.
- UI should stay in screen space.

In the Inspector:

```
CanvasLayer  
└─ MessageLabel (Label)
```

MessageLabel

text: "Find the interactive cube"

Update UI from the interaction

Modify interactive_cube.gd

```
@export var message_label: Label

func _on_area_3d_body_entered(body):
    if body.name == "Player":
        $MeshInstance3D.scale = Vector3.ONE * 1.5
        if message_label:
            message_label.text = "You found the cube!"
```

In the Inspector:

- Select InteractiveCube
- Find Message Label export
- Drag MessageLabel into the field